

EngagePro Chatbot

Project Notebook

Project Overview

Project: EngagePro Chatbot

Objective: Develop an interactive chatbot capable of engaging in natural language conversations and responding intelligently to user queries.

Development Journal

Stage 1 – Assignment Analysis

Objective

Analyse the assignment requirements and identify the scope, deliverables and assessment criteria before beginning development.

Assignment Requirements

- Develop an interactive chatbot capable of engaging in natural language conversations.
- Respond intelligently to user queries.
- Apply ethical safeguards to minimise hallucinations and bias.
- Demonstrate the application during a live presentation.
- Submit a technical report describing the design and implementation.

Key Decisions

- Build the application using the modern LangChain ecosystem.
- Prioritise a simple, maintainable architecture over unnecessary features.
- Implement Retrieval-Augmented Generation (RAG) using the company brochure.
- Add advanced capabilities only if they provide clear value.

Success Criteria

The chatbot should:

- Answer general conversational questions.
- Answer questions using the EngagePro company brochure.
- Produce factual responses grounded by retrieved documents.
- Demonstrate clean software engineering practices.
- Be easy to explain during the presentation.

Outcome

Project scope approved.

Stage 2 – System Architecture

Stage 2 – System Architecture

Objective

Design a modular architecture that is simple, maintainable and aligned with the modern LangChain ecosystem.

Architecture

User

↓

Streamlit User Interface

↓

LangGraph Workflow

↓

Intent Routing (future)

↓

General Chat OR RAG

↓

OpenAI GPT

↓

Response

Design Principles

- Separate UI from business logic.
- Use LangGraph as the workflow engine.
- Store configuration centrally in `config.py`.
- Keep each module focused on a single responsibility.
- Build the application incrementally.

Ethical Safeguards

- Use Retrieval-Augmented Generation (RAG) to ground responses in the company brochure.
- Clearly separate company knowledge from general knowledge.

- Keep model temperature low to reduce hallucinations.
- Provide document source information where appropriate.
- Avoid generating information not supported by retrieved context.

Outcome

Architecture approved.

Stage 3 – Technology Stack

Objective

Select technologies that satisfy the assignment requirements while remaining simple, maintainable and easy to justify.

Selected Technologies

Component	Technology	Reason
Programming Language	Python 3.11	Stable, widely supported by AI libraries
UI	Streamlit	Simple and rapid web application development
Workflow	LangGraph	Recommended workflow framework for LangChain v1
LLM Framework	LangChain v1	Modern ecosystem with modular architecture
LLM	OpenAI GPT	Reliable conversational performance
Future Local LLM	LM Studio	Planned optional support through LLM factory
Vector Database	ChromaDB	Local, lightweight, no cloud dependency
Embedding Model	OpenAI Embeddings	Seamless integration with LangChain
PDF Loader	PyPDFLoader	Native LangChain document loader
Text Splitter	RecursiveCharacterTextSplitter	Preserves context with configurable overlap

Technologies Considered

- Pinecone
- MongoDB Atlas Vector Search
- Classic LangChain chains

Reasons for Rejection

Pinecone

- Requires cloud infrastructure.
- Unnecessary for a single brochure.

MongoDB Atlas

- More complex than required.

Classic LangChain Chains

- Less flexible than LangGraph for future enhancements.

Outcome

Technology stack approved.

Iteration 1 – Environment Setup

Objective

Prepare the development environment and verify that the core technologies work together before building the chatbot.

Environment

- Created Conda environment:
 - EngageProChatbot
- Python Version:
 - 3.11.15

Project Structure Created

- app.py
- config.py
- llm/
- graph/
- rag/
- scripts/
- tests/
- prompts/
- vectorstore/
- docs/

Core Packages Installed

- LangChain v1
- LangGraph
- Streamlit
- OpenAI
- python-dotenv

Files Created

- config.py

- llm/llm_factory.py
- graph/state.py
- graph/nodes.py
- graph/workflow.py
- scripts/test_llm.py
- scripts/test_graph.py

Key Design Decisions

- Use Conda instead of a standard virtual environment.
- Use Python 3.11 for compatibility with LangChain.
- Implement an LLM Factory to support future switching between OpenAI and LM Studio.
- Adopt LangGraph's `MessagesState` instead of a custom state.
- Develop using small, testable increments.

Testing

- ✅ OpenAI API connection verified. ✅ LangChain working. ✅ LangGraph workflow executed successfully.
- ✅ GPT response generated through LangGraph.

Lessons Learned

- Run project modules using:

```
python -m module_name
```

instead of

```
python file.py
```

- MessagesState provides a future-proof conversation model.
- Verifying each layer independently simplifies debugging.

Status

- ✅ Completed

Git Commit

feat: initialise project environment and LangGraph foundation

Iteration 2 – Streamlit MVP

Objective

Build a working chatbot UI using Streamlit and integrate it with LangGraph.

Files Created

- app.py

Files Modified

- graph/state.py
- graph/nodes.py
- llm/llm_factory.py

Key Design Decisions

- Refactored to MessagesState.
- Used Streamlit Session State.
- Separated UI from workflow.

Testing

✅ Streamlit launched. ✅ Chat interface working. ✅ GPT responses generated. ✅ Conversation history preserved.

Lessons Learned

- Streamlit reruns the script after each interaction.
- Session State preserves conversation history.
- MessagesState is the recommended LangGraph state model.
- Streamlit Session State is essential for maintaining conversation history.
- Separating the UI from the workflow improves maintainability.
- Incremental development reduces debugging complexity.

Git Commit

feat: build Streamlit chatbot with LangGraph integration (To be completed)

Iteration 3.1 – PDF Loading

Objective

Create the first component of the RAG pipeline by successfully loading the EngagePro company brochure.

Files Created

- rag/ingest.py

Files Modified

- config.py (added BROCHURE_FILE if implemented)

Key Design Decisions

- Used PyPDFLoader to load PDF documents.

- Separated document loading into a dedicated function (`load_documents()`).
- Stored the brochure path in the configuration rather than hard-coding it.
- Chose to validate PDF loading before implementing chunking or embeddings.

Testing

- Successfully loaded 8-page brochure.
- Verified readable text extraction.
- Confirmed the first page preview displayed correctly.

Lessons Learned

- RAG development should begin by validating the source documents.
- PyPDFLoader extracts each page as a LangChain `Document`.
- Testing each stage independently simplifies debugging.

Status

✅ Completed

Iteration 3.2 – Document Chunking

Objective

Split the EngagePro brochure into coherent, overlapping chunks suitable for semantic retrieval.

Files Created

- `rag/chunking.py`
- `scripts/test_chunking.py`

Files Modified

- `config.py`

Key Design Decisions

- Used `RecursiveCharacterTextSplitter`.
- Moved `CHUNK_SIZE` and `CHUNK_OVERLAP` into `config.py`.
- Chose a chunk size of **1200 characters** with **200 characters overlap** after inspecting the brochure.
- Separated chunking logic into its own module following the Single Responsibility Principle.

Testing

- ✅ Successfully split the 8-page brochure into 15 chunks.
- ✅ Verified chunk sizes were consistent.
- ✅ Confirmed chunk overlap preserved context.

Engineering Rationale

Rather than adopting default chunking parameters from tutorials, the brochure was inspected first to understand its structure. The selected chunk size produced coherent chunks that preserved paragraph boundaries while maintaining enough overlap to improve semantic retrieval.

Lessons Learned

- Chunking quality has a significant impact on retrieval performance.
- Overlap helps preserve context between adjacent chunks.
- Inspecting chunk output before generating embeddings leads to better design decisions.

Status

✅ Completed

Iteration 3.3 – Knowledge Base Construction

Objective

Create a persistent vector database from the EngagePro company brochure for semantic retrieval.

Files Created

- rag/embeddings.py
- rag/vectorstore.py
- scripts/test_vectorstore.py

Files Modified

- config.py
- rag/ingest.py

Key Design Decisions

- Centralised embedding model configuration in `config.py`.
- Separated embedding generation from vector database management.
- Used ChromaDB as a local persistent vector store.
- Persisted embeddings to disk so indexing only needs to occur when source documents change.

Testing

✅ Embedding model successfully created. ✅ ChromaDB created successfully. ✅ 15 document chunks indexed. ✅ Vector database persisted locally. ✅ Semantic retrieval successfully returned relevant brochure content.

Engineering Rationale

The indexing pipeline is separated from runtime retrieval. This avoids regenerating embeddings whenever the chatbot starts, improving efficiency and reducing API usage.

Lessons Learned

- Embeddings transform text into semantic vectors for similarity search.
- ChromaDB stores vectors locally, allowing the chatbot to reuse the knowledge base without regenerating embeddings.
- Retrieval quality depends heavily on document quality and chunking strategy.
- Separating indexing from runtime retrieval improves efficiency and maintainability.

Status

✅ Completed

Definition of Done

- ✅ PDF loaded successfully.
- ✅ Document inspected.
- ✅ Metadata inspected.
- ✅ Document chunked.
- ✅ Chunk quality evaluated.
- ✅ Embeddings generated.
- ✅ ChromaDB created and persisted.
- ✅ Semantic retrieval validated.

Iteration 3 completed successfully.

Iteration 4.1 – RAG Integration

Objective

Integrate the knowledge base into the LangGraph chatbot so that responses are grounded in the EngagePro brochure.

Files Created

- rag/retrieval.py
- prompts/rag_prompt.py
- scripts/test_prompt.py
- scripts/test_rag.py

Files Modified

- graph/nodes.py

Key Design Decisions

- Kept the LangGraph workflow unchanged.
- Integrated retrieval inside the existing chat node.
- Separated retrieval, prompt construction and LLM invocation into dedicated modules.
- Used prompt engineering to reduce hallucinations.

Testing

- ✓ Retrieval returned relevant brochure chunks.
- ✓ Prompt constructed successfully.
- Chatbot integration (pending).

Engineering Rationale

Rather than adding additional LangGraph nodes, retrieval was integrated into the existing chat node. This keeps the workflow simple while preserving modularity through dedicated retrieval and prompt modules.

Testing

- ✓ Retrieved relevant brochure documents.
- ✓ Prompt successfully constructed.
- ✓ LangGraph integrated with the RAG pipeline.
- ✓ Chatbot answered EngagePro questions using brochure content.
- ✓ Chatbot correctly refused to answer questions not supported by the brochure.

Lessons Learned

- Retrieval quality and prompt design both influence RAG performance.
- An overly restrictive prompt may prevent the model from synthesising relevant information.
- Prompt engineering can improve answer quality without changing the retrieval pipeline.
- Modular design allowed RAG integration by modifying only the chat node.

Status

- ✓ Completed

Iteration 5.1 – Lightweight Routing Agent

Objective

Implement a lightweight routing agent that classifies user questions before they are processed by the chatbot.

Files Created

- routing/router.py
- routing/**init**.py
- scripts/test_router.py

Key Design Decisions

- Used the existing GPT model as an intent classifier.
- Restricted the classifier to two outputs: **engagepro** and **general**.
- Separated routing logic into its own module to keep the LangGraph nodes focused on orchestration.

Engineering Rationale

A lightweight routing agent improves the chatbot's usability by ensuring that only EngagePro-related questions are sent through the RAG workflow. General and technical questions can be routed to an independent processing branch. This architecture enables EngagePro-specific questions to use Retrieval-Augmented Generation (RAG), while general knowledge questions are answered using information retrieved from Wikipedia. The routing agent demonstrates the use of an LLM-based agent without introducing unnecessary complexity.

Testing

- ✓ Routing agent correctly classified all evaluation questions.

Lessons Learned

- LLMs can perform reliable intent classification using carefully designed prompts.
- Restricting the classifier to two valid outputs improved consistency.
- Separating routing from response generation follows the Single Responsibility Principle and simplifies the LangGraph workflow.

Status

- ✓ Completed

Iteration 5.2 – Conditional Workflow Routing

Objective

Integrate the routing agent into the LangGraph workflow so that EngagePro-related questions are processed using Retrieval-Augmented Generation (RAG), while general and technical questions are routed to a separate processing branch.

Files Modified

- graph/workflow.py
- graph/nodes.py
- graph/state.py
- scripts/test_graph.py

Key Design Decisions

- Used LangGraph conditional edges to implement dynamic routing.
- Added a `route` field to `ChatState` to store the routing decision.
- Split the chatbot into two independent processing nodes:
 - `rag_chat_node()`
 - `general_chat_node()`
- Kept the router responsible only for classification to maintain separation of concerns.

Engineering Rationale

The chatbot architecture was redesigned to follow a modular workflow. Rather than allowing every question to follow the same execution path, the routing agent first determines the question type before directing it to the appropriate processing node. This improves scalability and allows additional routes to be added in future without modifying the existing nodes.

Testing

- ✓ LangGraph workflow compiled successfully.
- ✓ EngagePro questions routed to the RAG branch.
- ✓ General questions routed to the general knowledge branch.

Lessons Learned

- LangGraph conditional routing simplifies complex workflows.
- Keeping routing logic separate from response generation improves maintainability.
- Modular node design makes the workflow easier to extend.

Status

- ✓ Completed

Iteration 5.3 – Wikipedia Integration

Objective

Implement Wikipedia search for general and technical questions to satisfy the assignment requirement that the chatbot retrieve factual information from Wikipedia before generating a response.

Files Created

- `wiki/init.py`
- `wiki/wikipedia_search.py`
- `prompts/wiki_prompt.py`
- `scripts/test_wikipedia.py`

Files Modified

- `graph/nodes.py`
- `requirements.txt`

Key Design Decisions

- Used the official Wikipedia REST API instead of a third-party wrapper library.
- Added a query preprocessing function (`clean_query()`) to convert natural-language questions into effective Wikipedia search queries.
- Introduced a dedicated Wikipedia prompt builder to ensure responses were grounded only in retrieved information.
- Continued using GPT to summarise and present Wikipedia content conversationally.

Engineering Rationale

Rather than relying on the language model's internal knowledge, the chatbot retrieves relevant Wikipedia information first and uses it as context for response generation. This reduces hallucinations, improves factual grounding, and satisfies the assignment requirement for Wikipedia-based knowledge retrieval while preserving the modular LangGraph architecture.

Testing

- ✅ Wikipedia REST API successfully retrieved article summaries.
- ✅ Query preprocessing improved search accuracy for natural-language questions.
- ✅ General and technical questions were correctly answered using Wikipedia.
- ✅ End-to-end Streamlit testing confirmed successful routing between RAG and Wikipedia branches.

Lessons Learned

- Official REST APIs are often more reliable than older wrapper libraries.
- Natural-language queries should be preprocessed before performing information retrieval.
- Prompt engineering plays an important role in ensuring grounded responses and reducing hallucinations.
- Separating retrieval, prompt construction, and response generation improves maintainability and readability.

Status

- ✅ Completed

Architecture Decisions (ADR)

Architecture Decisions (ADR)

ADR-001: Adopt LangGraph as the Workflow Framework

Status

Accepted

Decision

Use LangGraph to orchestrate the chatbot workflow.

Context

The project required multiple processing stages, including question routing, Retrieval-Augmented Generation (RAG), and Wikipedia retrieval. A workflow framework was needed to coordinate these components while

remaining modular and extensible.

Alternatives Considered

- Sequential Python function calls
- LangChain Chains
- LangGraph

Rationale

LangGraph provides a graph-based workflow that cleanly separates individual processing nodes while supporting conditional routing. This architecture improves maintainability and allows additional processing branches to be added in future with minimal code changes.

Consequences

- Clear separation of responsibilities.
 - Modular workflow design.
 - Easier future expansion.
-

ADR-002: Use Retrieval-Augmented Generation (RAG) for EngagePro Knowledge

Status

Accepted

Decision

Use Retrieval-Augmented Generation instead of relying solely on the language model's internal knowledge.

Context

The chatbot must answer questions specifically about EngagePro using information contained in the provided company brochure.

Alternatives Considered

- Prompt the LLM with the entire brochure.
- Fine-tune an LLM.
- Retrieval-Augmented Generation (RAG).

Rationale

RAG retrieves only the most relevant document chunks for each user question, reducing context size while improving response accuracy and factual grounding.

Consequences

- Reduced hallucinations.

- Better scalability for larger knowledge bases.
 - Efficient prompt construction.
-

ADR-003: Store Embeddings in ChromaDB

Status

Accepted

Decision

Use ChromaDB as the vector database.

Context

The project required persistent storage of document embeddings to support semantic search.

Alternatives Considered

- FAISS
- Pinecone
- ChromaDB

Rationale

ChromaDB is lightweight, easy to integrate, open source, and suitable for a local educational project without requiring cloud infrastructure.

Consequences

- Simple deployment.
 - Persistent local storage.
 - Minimal configuration.
-

ADR-004: Implement an LLM-Based Routing Agent

Status

Accepted

Decision

Introduce a lightweight routing agent to classify user questions before processing.

Context

The chatbot needed to distinguish between EngagePro-specific questions and general or technical questions.

Alternatives Considered

- Keyword matching
- Rule-based routing
- LLM-based classification

Rationale

An LLM provides flexible intent classification without maintaining complex keyword lists. Restricting the output to two valid routes (**engagepro** and **general**) improved consistency.

Consequences

- Improved modularity.
 - Better separation of workflows.
 - Simplified future expansion.
-

ADR-005: Use Wikipedia as the General Knowledge Source

Status

Accepted

Decision

Retrieve general and technical information from Wikipedia before generating responses.

Context

The assignment required the chatbot to answer general or technical questions using Wikipedia.

Alternatives Considered

- Use the LLM's internal knowledge.
- Internet search engines.
- Wikipedia REST API.

Rationale

Wikipedia provides a freely accessible, well-known knowledge source. Retrieving information before prompting the LLM helps reduce hallucinations and satisfies the assignment requirements.

Consequences

- Grounded responses.
 - Improved factual accuracy.
 - Clear separation between company knowledge and public knowledge.
-

ADR-006: Use Prompt Engineering to Reduce Hallucinations

Status

Accepted

Decision

Design separate prompts for RAG and Wikipedia responses.

Context

The chatbot needed to minimise hallucinations and encourage grounded responses.

Alternatives Considered

- Single generic prompt.
- Separate prompts for each knowledge source.

Rationale

Dedicated prompts allow instructions to be tailored to each retrieval source while encouraging the language model to respond honestly when insufficient information is available.

Consequences

- Improved response quality.
- Reduced unsupported answers.
- Easier prompt maintenance.

Testing Log

Testing Log

Test ID	Component	Test Description	Result
T01	PDF Loading	Verify the EngagePro brochure can be loaded successfully.	Pass
T02	Document Chunking	Verify the brochure is split into overlapping chunks correctly.	Pass
T03	Embeddings	Verify embeddings are generated using the OpenAI embedding model.	Pass
T04	ChromaDB	Verify document chunks are indexed into the vector database.	Pass
T05	Retrieval	Verify relevant document chunks are retrieved for EngagePro questions.	Pass

Test ID	Component	Test Description	Result
T06	RAG Prompt	Verify retrieved documents are correctly inserted into the RAG prompt.	✓ Pass
T07	Streamlit Interface	Verify the chatbot interface launches and accepts user input.	✓ Pass
T08	LangGraph Workflow	Verify the LangGraph workflow executes successfully.	✓ Pass
T09	Routing Agent	Verify questions are correctly classified into engagepro and general .	✓ Pass
T10	Conditional Routing	Verify questions are routed to the correct processing node.	✓ Pass
T11	Wikipedia Retrieval	Verify Wikipedia summaries are successfully retrieved using the REST API.	✓ Pass
T12	End-to-End Integration	Verify the chatbot answers both EngagePro-specific and general questions correctly.	✓ Pass

Functional Test Cases

EngagePro Knowledge (RAG)

Question	Expected Result	Status
What is EngagePro's mission?	Answer generated from brochure	✓
What products does EngagePro provide?	Answer generated from brochure	✓
Where is EngagePro located?	Correct location returned	✓

Wikipedia Knowledge

Question	Expected Result	Status
What is Artificial Intelligence?	Wikipedia-based response	✓
Explain Retrieval-Augmented Generation.	Wikipedia-based response	✓
What is LangChain?	Wikipedia-based response	✓

Routing

Question	Expected Route	Status
What is EngagePro's mission?	RAG	✓
Explain Artificial Intelligence.	Wikipedia	✓

Question	Expected Route	Status
What services does EngagePro offer?	RAG	✓
Explain Retrieval-Augmented Generation.	Wikipedia	✓

Lessons Learned

Lessons Learned

Throughout the development of the EngagePro Chatbot, several important technical and software engineering lessons were learned.

1. Incremental Development Simplifies Complex Projects

Developing the chatbot in small, testable iterations made debugging significantly easier. Each component was implemented and validated independently before integration into the complete system.

2. Modular Architecture Improves Maintainability

Separating the project into dedicated modules (RAG, routing, prompts, Wikipedia retrieval, LangGraph workflow, and user interface) made the codebase easier to understand, maintain, and extend.

3. Retrieval-Augmented Generation Produces More Reliable Responses

Using Retrieval-Augmented Generation (RAG) enabled the chatbot to answer EngagePro-specific questions using information retrieved directly from the company brochure rather than relying solely on the language model's internal knowledge. This reduced hallucinations and improved response accuracy.

4. Prompt Engineering Has a Significant Impact

Carefully designed prompts greatly improved response quality. Separate prompts for EngagePro knowledge and Wikipedia retrieval encouraged the language model to generate grounded responses and acknowledge when sufficient information was unavailable.

5. Workflow Orchestration Simplifies Complex Logic

LangGraph provided a clear mechanism for organising the chatbot workflow. The routing node, RAG node, and Wikipedia node each had distinct responsibilities, resulting in a clean and extensible architecture.

6. Systematic Testing Improves Reliability

Testing each module independently before integrating it into the complete application helped identify issues early. Dedicated test scripts simplified debugging and increased confidence in the final implementation.

7. Git Version Control Supports Incremental Development

Using Git commits and version tags throughout the project provided clear development milestones and made it easier to track progress and maintain a stable codebase.

8. AI-Assisted Development Accelerates Learning

ChatGPT served as a technical mentor and learning companion throughout the project by explaining concepts, reviewing architecture, assisting with debugging, and discussing design alternatives. This iterative collaboration improved both the quality of the implementation and the author's understanding of Retrieval-Augmented Generation, LangGraph, prompt engineering, and modern LLM application development.

Presentation Notes

Demonstration Flow

The live demonstration will follow the chatbot's workflow from user input to response generation:

1. Introduce the project objective.
2. Explain the overall system architecture.
3. Demonstrate an EngagePro-specific query using RAG.
4. Demonstrate a general knowledge query using Wikipedia retrieval.
5. Explain the routing agent and LangGraph workflow.
6. Summarise key engineering decisions and future improvements.

Key Points

- Demonstrate both chatbot knowledge sources.
 - Explain how the routing agent selects the appropriate workflow.
 - Highlight prompt engineering techniques used to reduce hallucinations.
 - Emphasise the modular architecture and incremental development approach.
-

Future Improvements

Although the chatbot satisfies the assignment requirements, several enhancements could further improve its capabilities and usability.

1. Support Multiple Knowledge Sources

Extend the RAG pipeline to index multiple company documents instead of relying on a single brochure. This would enable the chatbot to answer a wider range of business-related questions.

2. Source Citation

Display the document source and page number used to generate each RAG response. This would improve transparency and allow users to verify the information provided.

3. Persistent Conversation History

Store conversation history in a database so that users can resume previous chat sessions across multiple visits.

4. Additional Knowledge Sources

Integrate other trusted external knowledge sources, such as company documentation or enterprise knowledge bases, alongside Wikipedia.

5. Enhanced User Authentication

Introduce user authentication and role-based access control to support personalised experiences and protect sensitive company information.

6. Containerised Deployment

Package the chatbot using Docker to simplify deployment and improve portability across different environments.

7. Cloud Deployment

Deploy the chatbot to a cloud platform such as Microsoft Azure, AWS, or Google Cloud Platform to improve scalability and availability.

8. Continuous Evaluation

Introduce automated evaluation metrics to monitor response quality, retrieval accuracy, and user satisfaction over time.